ACTIVITAT AVALUABLE AC2
Mòdul: M0612- Desenvolupament web en entorn client **U1 – Fonaments de JavaScript** **Professor:** Albert Guardiola **Data límit d'entrega:** 30/09/2026 23:59 **Mètode d'entrega:** Per mitjà del Clickedu de l'assignatura. **Instruccions:** S'ha d'entregar un únic document amb el nom: ***MP0616-U1-AC2-Nom_Alumne.js*** **Avaluació de la tasca:** La nota serà la del control posterior que es realitzarà a classe.

Resultats de l'aprenentatge:
RA1, RA2, RA3, RA4

AC2 - Gestor de Reserves amb preferències d'usuari

1. Context Professional

En plataformes globals de reserves (hoteleria, lloguer de vehicles o ticketing), la gestió temporal i l'atenció a la configuració regional de l'usuari determinat la viabilitat de l'aplicació.

Un mal ús de l'objecte Date pot provocar **errors de facturació per càlcul incorrecte de dates**, desfasaments de dia per culpa del *Timezone Offset* o reserves en dates passades. A més, no desar les preferències d'usuari de forma segura a *localStorage* (o no reaccionar als canvis de configuració entre pestanyes) corromp l'experiència d'usuari.

Finalment, les aplicacions web modernes han d'oferir experiències equiparables a les aplicacions natives: compartir reserves amb un clic, enviar alertes de proximitat, copiar localitzadors, adaptar-se a l'idioma/moneda del client i estalviar bateria i dades pausant les peticions quan la pestanya no està visible.

2. Blocs d'Exercicis de Refactorització

Bloc 1: Gestió Temporal Avançada, Zones Horàries i Internacionalització (Date & Intl)

Temari cobert: Instanciació de Date, manipulació de mil·lisegons, desfasament horari (*Timezone Offset*), formatació regional de dates (Intl.DateTimeFormat), formatació de monedes (Intl.NumberFormat), validació d'ordres temporals i formats d'emmagatzematge estables (ISO 8601).

Exercici 1.1: Operacions amb Dates, Contaminació NaN i el Trap del *Timezone Offset*

- **Diagnòstic de l'error:** A la funció calculateStay(), es produeixen tres errors greus en la gestió temporal de l'aplicació:
  1. **Operació matemàtica invàlida (NaN):** La línia var nights = checkOutStr - checkInStr; intenta restar directament dues cadenes de text (string). Com que JavaScript no pot convertir cadenes com "2026-09-18" a valors numèrics per la presència dels guions, la conversió falla i retorna NaN (*Not a Number*), contaminant la resta d'operacions financeres (NaN * preu = NaN).
  2. **Avaluació alfabètica de cadenes:** La condició checkInStr >= checkOutStr compara cadenes de text en lloc d'objectes data. Tot i que en el format YYYY-MM-DD pot semblar que funciona, si l'entrada canvia a formats locals (ex: "02/10/2026" >= "15/09/2026"), la comparació alfabètica dona true tot i ser falsa temporalment.
  3. **Desfasament pel fus horari (UTC vs Local):** Instanciar cadenes en format ISO directament amb new Date("2026-09-18") fa que el motor de JS interpreti la data a la mitjanit UTC (00:00:00Z). En convertir-se a l'hora local d'usuaris en fusos horaris de l'Oest (com Amèrica / UTC-4), la data retrocedeix al dia anterior (2026-09-17 20:00:00), mostrant un dia incorrecte a la interfície.

- **Com ho pots comprovar? (Prova pràctica):**
  1. **Comprovació del bug NaN:** Amb el codi de partida, deixa les dates per defecte i prem el botó **Calcular Preu Estada**. Observa com la interfície mostra **NaN nits** i un preu total de **NaN €**.
  2. **Comprovació de la lògica d'ordre:** Canvia les dates posant una data de sortida (15/09/2026) anterior a la d'entrada (18/09/2026). Prem el botó de calcular i comprova com el programa permet l'operació sense llançar cap alerta o dona resultats absurds.
  3. **Comprovació de la conversió ISO:** Obre la consola del navegador (F12) i executa aquestes dues línies per veure la diferència d'hores:
     - new Date("2026-09-18").toString() → Observa que marca les 02:00:00 (ha afegit +2 hores del fus d'estiu CEST sobre la mitjanit UTC).
     - new Date(2026, 8, 18).toString() → Observa que marca les 00:00:00 (mitjanit exacta en hora local, recordant que el mes 8 correspon a setembre).

- **Tasca pràctica (Pas a pas):**
  1. **Parseig segur en hora local:** Implementa una funció o lògica que desmunti la cadena "YYYY-MM-DD" en valors enters (any, mes, dia) i instanciï la data fent servir new Date(year, month - 1, day) per forçar que la data es creï en l'hora local de l'ordinador sense cap desfasament UTC:

```
function parseLocalDate(dateString) {
      // Desmuntem "2026-09-18" -> [2026, 9, 18]
      const [year, month, day] = dateString.split('-').map(Number);

     // Instanciem forçant hora local: (2026, 8, 18)
     return new Date(year, month - 1, day);}
```

     Fes servir aquesta funció a *calculateStay()* per parsejar les dates correctament:

```
const checkIn = parseLocalDate(checkInStr);
const checkOut = parseLocalDate(checkOutStr);
```

  2. **Validació d'errors i ordre temporal:** A la funció, *calculateStay(),* utilitza Number.isNaN(date.getTime()) per comprovar que les dates siguin vàlides i verifica que la data de sortida sigui estrictament posterior a la d'entrada (checkOut <= checkIn). Si la comprovació falla, interromp l'execució immediatament llançant un error personalitzat:

```
// 1. Validar que siguin dates numèricament vàlides
if (Number.isNaN(checkIn.getTime()) || Number.isNaN(checkOut.getTime())) {
      throw new Error("Si us plau, introdueix dues dates vàlides."); } /

/ 2. Validar l'ordre temporal (comparació directa entre objectes Date)
if (checkOut <= checkIn) {
     throw new Error("La data de sortida ha de ser estrictament posterior a la data
d'entrada."); }
```

     Implementa una estructura *catch…try* que reculli aquest error i mostri un alert amb el missatge.

  3. **Càlcul matemàtic en mil·lisegons:** Ara que les dates *checkIn* i *checkOut* són dates, no es poden restar drectament: obtén la diferència numèrica en mil·lisegons amb .getTime() i divideix el resultat pel nombre de mil·lisegons que té un dia (1 dia $= 1000$ ms $\times 60$ s $\times 60$ m $\times 24$ h $= 86.400.000$ ms):

```
const diffTime = checkOut.getTime() - checkIn.getTime();
const MS_PER_DAY = 1000 * 60 * 60 * 24;

const nights  = diffTime / MS_PER_DAY;
```

  4. **Protecció contra canvis d'horari (DST):** Aplica Math.round() al resultat de la divisió (Math.round(diffTime / MS_PER_DAY)) per evitar decimals erronis (2.95 o 3.04) causats pel canvi d'horari oficial d'estiu/hivern.

Exercici 1.2: Internacionalització i Formatació Regional amb *Intl.DateTimeFormat* i *Intl.NumberFormat*

- **Diagnòstic de l'error:** A les funcions `formatBookingDate()` i `formatPrice()`, la visualització de dades pateix mancances greus d'adaptabilitat regional:
    1. **Concatenació manual i desfasament de mesos:** La data es construeix concatenant cadenes manualment:

       ```
       d.getDate() + "/" + d.getMonth() + "/" + d.getFullYear().
       ```

       Això provoca dos errors: oblida que `getMonth()` retorna un índex basat en zero (`0` per a gener, `8` per a setembre), mostrant setembre com a mes `8`, i imposa un format rígid (DD/MM/YYYY) que ignora la configuració regional de l'usuari (com el format EUA `MM/DD/YYYY`).

    2. **Formatació financera deficient:** La funció `formatPrice()` s'limita a afegir el símbol `" €"` al final del número (`amount + " €"`). Això ignora la moneda triada per l'usuari (`USD`, `GBP`, etc.), no aplica separadors de milers/decimals adequats segons el país i posa el símbol de la moneda sempre a la dreta (mentre que en dòlars o lliures ha d'anar a l'esquerra: `$120.50`).

- **Com ho pots comprovar? (Prova pràctica):**
    1. **Comprovació del mes incorrecte:** Amb el codi de partida, selecciona una data d'entrada al setembre (ex: `18/09/2026`). Prem el botó **Calcular Preu Estada** i observa com al resum visual la data d'entrada es mostra com a `18/8/2026` (agost) en lloc de `18/9/2026`.
    2. **Comprovació de la moneda rígida:** Al desplegable de preferències, canvia la moneda a **USD** o **GBP**. Prem el botó de calcular i comprova com el preu total continua mostrant el símbol `€` al final sense fer la conversió de format regional visual.

- **Tasca pràctica (Pas a pas):**
    1. **Formatació regional de dates amb `Intl.DateTimeFormat`:** Refactoritza la funció `formatBookingDate(dateInput, locale)` per utilitzar l'API nativa d'internacionalització. Defineix opcions per mostrar el dia en dos dígits, el mes amb nom complet i l'any en quatre dígits:

       ```
       function formatBookingDate(dateInput, locale = "ca-ES") {
           const date = typeof dateInput === "string" ?
       parseLocalDate(dateInput) : dateInput;

           if (Number.isNaN(date.getTime())) {
               return "Data invàlida";
           }

           const formatter = new Intl.DateTimeFormat(locale, {
               day: "2-digit",
               month: "long",
               year: "numeric"
           });

           return formatter.format(date);
       }
       ```

2. **Formatació financera amb `Intl.NumberFormat`:** Refactoritza `formatPrice(amount, currency, locale)` passant-li tant el codi d'idioma/país (`locale`) com el codi ISO de la moneda (`currency`). Utilitza l'estil `currency` per delegar al navegador la col·locació del símbol, els separadors decimals i de milers:

```
function formatPrice(amount, currency = "EUR", locale = "ca-
ES") {
    if (typeof amount !== "number" || Number.isNaN(amount)) {
        return "0.00 €";
    }

    return new Intl.NumberFormat(locale, {
        style: "currency",
        currency: currency
    }).format(amount);
}
```

3. **Integració amb les preferències de l'usuari:** Assegura't que el controlador de la interfície llegeixi l'idioma (`lang`) i la moneda (`currency`) triats a les preferències de l'usuari i els passi com a paràmetres a `formatBookingDate` i `formatPrice` en refrescar la vista.

   És a dir, assegura't que a la funció *handleCalculateStay* es llegeixen els *select* de idioma i mondea de la interfície:

```
const currentLang =
document.querySelector("#langSelect").value;
const currentCurrency =
document.querySelector("#currencySelect").value;
```

   i es passen a les funcions formatadores que acabes de refactoritzar.

Bloc 2: Persistència de Preferències i Sincronització Inter-pestanya

Exercici 2.1: Emmagatzematge d'Estructures Complexes a `localStorage` i Gestió del Tema Visual

- **Diagnòstic de l'error:** A les funcions `savePreferences()`, `loadPreferences()` i en la gestió del tema de la interfície es detecten tres fallades estructurals:
    1. **Serialització defectuosa (`[object Object]`):** La funció `savePreferences()` intenta desar un objecte JavaScript directament amb `localStorage.setItem("user_preferences", preferences)`. Com que `localStorage` només admet cadenes de text (*strings*), el navegador converteix l'objecte implícitament cridant a `.toString()`, desant la cadena inútil `"[object Object]"`.
    2. **Accés a cadenes no parsejades:** A `loadPreferences()`, el codi intenta accedir a `prefs.theme` sobre el resultat directe de `localStorage.getItem()`. Si la dada no es converteix de nou a objecte amb `JSON.parse()`, l'accés a les propietats retorna `undefined`.
    3. **Absència de reactivitat i aplicació del tema visual:** Al codi de partida, en seleccionar el mode clar o fosc al desplegable `#themeSelect`, **la interfície no fa cap canvi visual**. Falta tant un escoltador d'esdeveniments (`change`) per reaccionar a l'instant com la lògica que apliqui o treogui la classe CSS corresponent al `<body>`.

- **Com ho pots comprovar? (Prova pràctica):**
    1. **Comprovació de la manca de canvi de tema:** A la interfície, selecciona el "Tema Fosc" al desplegable `#themeSelect`. Comprova com la pantalla manté els colors clars i no reacciona al canvi.
    2. **Comprovació de la corrupció a `localStorage`:** Prem el botó **Desar Preferències**. Obre la consola de desenvolupador (F12), ves a la pestanya **Application > Local Storage** i selecciona el teu domini local. Observa com el valor emmagatzemat sota la clau `user_preferences` és literalment la cadena `"[object Object]"`.
    3. **Comprovació del retorn `undefined`:** A la consola, executa `loadPreferences()`. Observa com retorna `undefined` o `null` a causa de la impossibilitat de llegir dades d'una cadena malmosa.

- **Tasca pràctica (Pas a pas):**
    1. **Creació de la funció d'aplicació del tema (`applyTheme`):** Dissenya una funció auxiliar que rebi la cadena `"dark"` o `"light"` i modifiqui les meves classes del `<body>` per commutar els colors de la interfície:

```
function applyTheme(theme) {
    if (theme === "dark") {
        document.body.classList.add("dark-theme");
    } else {
        document.body.classList.remove("dark-theme");
    }
}
```

2. **Reactivitat immediata al desplegable:** Enllaça un esdeveniment `change` al selector `#themeSelect` perquè la interfície canviï de color al moment, abans i tot de prémer el botó de desar:

```
const themeSelect = document.querySelector("#themeSelect");
if (themeSelect) {
    themeSelect.addEventListener("change", (e) => {
        applyTheme(e.target.value);
    });
}
```

3. **Persistència correcta amb `JSON.stringify`:** Refactoritza `savePreferences(prefs)` per convertir l'objecte de preferències en una cadena JSON vàlida abans de desar-lo a `localStorage`. Inclou un camp de marca temporal (`lastSearch`) per registrar quan s'ha fet el desament:

```
function savePreferences(preferences) {
    const payload = {
        theme: preferences.theme,
        lang: preferences.lang,
        currency: preferences.currency,
        lastSearch: new Date().toISOString()
    };
    localStorage.setItem("user_preferences",
JSON.stringify(payload));
}
```

4. **Lectura segura amb `JSON.parse` i *Fallback*:** Refactoritza `loadPreferences()` utilitzant un bloc `try...catch` per parsejar el JSON desat de forma segura. Si la dada no existeix o està corrupta, ha de retornar un objecte de preferències per defecte (*fallback*):

```
function loadPreferences() {
    const defaultPreferences = {
        theme: "light",
        lang: "ca-ES",
        currency: "EUR",
        lastSearch: null
    };

    try {
        const rawData =
localStorage.getItem("user_preferences");
        if (!rawData) return defaultPreferences;
        return JSON.parse(rawData);
    } catch (error) {
        console.warn("Error llegint preferences de
localStorage, usant valors per defecte:", error);
        return defaultPreferences;
    }
}
```

5. **Restauració en carregar la pàgina:** A l'inici de l'aplicació (`DOMContentLoaded`), crida `loadPreferences()`, actualitza el valor dels desplegables de la pantalla per sincronitzar-los amb la memòria i executa `applyTheme(prefs.theme)` perquè el tema persisteixi en recarregar la pàgina.

Exercici 2.2: Sincronització Inter-pestanya mitjançant l'Esdeveniment `storage`

- **Diagnòstic de l'error:** Si un usuari té l'aplicació oberta en dues pestanyes diferents del mateix navegador i canvia el tema o la moneda a la Pestanya A, la Pestanya B continua mostrant la configuració antiga fins que es recarrega manualment. Això crea inconsistències d'estat a la sessió de l'usuari.
- **Com ho pots comprovar? (Prova pràctica):**
  1. Obre l'aplicació en dues pestanyes adjacents del navegador (Pestanya A i Pestanya B).
  2. A la Pestanya A, canvia el tema a "Fosc" i prem **Desar Preferències**.
  3. Canvia immediatament a la Pestanya B sense recarregar-la. Observa com la Pestanya B continua en mode clar i amb els valors anteriors.
- **Tasca pràctica (Pas a pas):**
  1. Escolta l'esdeveniment global `storage` a l'objecte `window`. Aquest esdeveniment només s'activa a les pestanyes secundàries quan la pestanya principal modifica el `localStorage`:

```
window.addEventListener("storage", (event) => {
    // Verifiquem que el canvi prové de la clau de les nostres
preferències
    if (event.key === "user_preferences" && event.newValue) {
        try {
            const newPrefs = JSON.parse(event.newValue);

            // 1. Apliquem el nou tema visual a l'instant
            applyTheme(newPrefs.theme);

            // 2. Sincronitzem els desplegables de la pantalla
            document.querySelector("#themeSelect").value =
newPrefs.theme;
            document.querySelector("#langSelect").value =
newPrefs.lang;
            document.querySelector("#currencySelect").value =
newPrefs.currency;

            console.log("Preferències sincronitzades des d'una
altra pestanya.");
        } catch (e) {
            console.error("Error en sincronitzar pestanyes:",
e);
        }
    }
});
```

Bloc 3: Integració de Web APIs Natives del Navegador

Funció Auxiliar d'Interfície: Feedback Visual al Sistema (showApiFeedback)

NOTA: Per tal de mostrar missatges informatius, d'èxit o d'error a la interfície sense utilitzar blocs alert() síncrons, hi ha implementada una funció auxiliar que escrigui directament al contenidor #apiFeedback present a l'HTML:

Exercici 3.1: Portapapers i Compartició amb *Feature Detection* (`Clipboard` & `Web Share API`)

- **Diagnòstic de l'error:** A les funcions `copyBookingCode()` i `shareBooking()` es cometen dos errors d'arquitectura web moderna:
    1. **Ús de mètodes obsolets i síncrons:** `copyBookingCode()` fa servir `document.execCommand("copy")`, un mecanisme en desús (*deprecated*) que pot ser bloquejat de manera imprevisible per les polítiques de seguretat dels navegadors moderns i que s'executa de forma síncrona.
    2. **Absència de Feature Detection (*Asumpció d'API Entorn*):** `shareBooking()` invoca directament `navigator.share()` sense comprovar si l'API està disponible. En la majoria de navegadors d'escriptori (on la Web Share API no s'ha / no es pot implementar), l'execució falla immediatament llançant un error no controlat (`Uncaught TypeError: navigator.share is not a function`).

- **Com ho pots comprovar? (Prova pràctica):**
    1. **Comprovació del portapapers obsolet:** Prem **Copiar Localitzador**. Tot i que pot funcionar en alguns entorns, la consola o l'inspector d'API pot mostrar avisos de desús (*deprecation warnings*).
    2. **Comprovació de l'error de compartició:** Executa l'aplicació en un navegador d'escriptori (Chrome, Firefox o Edge a Windows/Linux/macOS). Prem el botó **Compartir Reserva** i observa com a la consola de desenvolupador (F12) apareix un error en vermell de tipus `TypeError`. Si no surt l'error, comprova a CanIUse que el teu navegador és compatible amb

- **Tasca pràctica (Pas a pas):**
    1. **Migració a `navigator.clipboard` asíncron:** Refactoritza `copyBookingCode()` utilitzant la Clipboard API basada en promeses (`async/await`). Gestiona la resposta informant l'usuari mitjançant un panell de retorn visual (*feedback*) a la UI:

```
async function copyBookingCode(code) {
    if (!navigator.clipboard) {
        showApiFeedback("El teu navegador no admet la còpia
automàtica al portapapers.", "error");
        return;
    }

    try {
        await navigator.clipboard.writeText(code);
        showApiFeedback(`Localitzador ${code} copiat al
portapapers!`, "success");
    } catch (error) {
```

```
        showApiFeedback("No s'ha pogut copiar el codi.",
"error");
        console.error("Error en copiar:", error);
    }
}
```

2. **Detecció de característiques (*Feature Detection*) a la Web Share API:** Abans de cridar a `navigator.share()`, comprova la seva existència. Si no està disponible (ex: en entorns d'escriptori), ofereix un mecanisme alternatiu (*fallback*), com copiar l'enllaç de la reserva automàticament:

```
async function shareBooking(bookingData) {
    const shareData = {
        title: `Reserva a ${bookingData.hotelName}`,
        text: `Tinc una reserva confirmada a
${bookingData.hotelName} (Ref: ${bookingData.id}).`,
        url: window.location.href
    };

    // Feature Detection
    if (navigator.share) {
        try {
            await navigator.share(shareData);
            showApiFeedback("Reserva compartida amb èxit!",
"success");
        } catch (error) {
            // Si l'usuari cancel·la el diàleg de compartició,
es gestiona sense petar
            if (error.name !== "AbortError") {
                showApiFeedback("Error en compartir la
reserva.", "error");
            }
        }
    } else {
        // Fallback per a navegadors que no suporten Web Share
        await copyBookingCode(window.location.href);
        showApiFeedback("Enllaç de reserva copiat al
portapapers (Compartició no disponible en aquest navegador).",
"info");
    }
}
```

Exercici 3.2: Demanar permisos de notificació (`Notification API`)

- **Diagnòstic de l'error:** La funció `findNearbyHotels()` incompleix els protocols de seguretat i privadesa del navegador:
    1. **Instantiació sense permís del sistema:** `sendProximityNotification()` intenta crear `new Notification(...)` directament. Si l'usuari no ha donat permís explícit prèviament (`Notification.permission !== "granted"`), el navegador bloqueja l'acció i llança una excepció per violació de seguretat.

- **Com ho pots comprovar? (Prova pràctica):**
    1. **Comprovació del bloqueig de notificacions:** Prem el botó **Activar Notificació de Proximitat**. Comprova com no ocorre res. L'aplicació ha de demanar permís explícit per poder fer servir l'API de Notificacions.
    2.

- **Tasca pràctica (Pas a pas):**
    1. **Gestió del cicle de permisos a `Notifications API`:** Refactoritza la funció per sol·licitar permís explícitament abans d'instanciar cap notificació, avaluant els tres estats possibles (`granted`, `denied`, `default`):

```
async function sendProximityNotification(bookingData) {
    if (!("Notification" in window)) {
        showApiFeedback("Aquest navegador no suporta
notificacions meitat sistema.", "error");
        return;
    }

    let permission = Notification.permission;

    if (permission === "default") {
        permission = await Notification.requestPermission();
    }

    if (permission === "granted") {
        new Notification("Grand Palace Resort", {
            body: `La teva reserva (${bookingData.id}) comença
aviat!`,
            icon: "/favicon.ico"
        });
        showApiFeedback("Notificació enviada correctament.",
"success");
    } else {
        showApiFeedback("Has denegat el permís per rebre
notificacions.", "warning");
    }
}
```

Exercici 3.3: Geolocalització Defensiva (`Geolocation API`)

- **Diagnòstic de l'error:** Al fitxer app_2.js original, la funció encarregada de la geolocalització (findNearbyHotels) presenta tres problemes principals d'arquitectura i robustesa:

    1. **Absència de callback d'error:** La crida a *navigator.geolocation.getCurrentPosition()* només inclou el primer paràmetre (la funció d'èxit). Quan un usuari denega els permisos, quan falla el senyal GPS o quan se supera el temps d'espera, l'API no troba cap funció per gestionar l'excepció, deixant la petició "penjada" en silenci i sense cap retroacció per a l'usuari.
    2. **Manca d'opcions de configuració (*PositionOptions*):** No s'especifiquen paràmetres de control com *timeout* o *enableHighAccuracy*. Si el navegador no aconsegueix obtenir la ubicació, l'aplicació pot quedar-se esperant de manera indefinida.
    3. **Falta d'informació a la interfície:** Ni la posició trobada ni els missatges d'error es canalitzen cap a l'element visual *#apiFeedback.*

- **Com ho pots comprovar? (Prova pràctica):**

  - **Comprovar la fallada del codi vell (Permís Denegat):**

    - Obre l'aplicació des de la teva pestanya del navegador i prem `F12` per obrir les eines de desenvolupador (DevTools).
    - Fes clic a la icona del candau/configuració a l'esquerra de la URL de la barra d'adreces.
    - Modifica el permís d'**Ubicació** a **Bloquejar** (Denegar).
    - Prem el botó **Cerca Hotels Propers (GPS)**: veuràs que no passa absolutament res, la pantalla no s'actualitza i la consola no informa de la denegació (el codi vell ha fallat en silenci).

  - **Tasca pràctica (Pas a pas):**

    Refactoritza la funció findNearbyHotels i la seva vinculació al DOM per incloure la verificació de suport, les opcions de configuració, el callback d'error defensiu amb un bloc switch i l'enviament de missatges a la interfície:

```
/**
 * Cerca la ubicació per GPS gestionant callbacks d'error,
opcions de temps i feedback visual.
 */
function findNearbyHotels() {
    // 1. Verificació de suport del navegador
    if (!("geolocation" in navigator)) {
        showApiFeedback("La geolocalització no està disponible
al teu navegador.", "error");
        return;
    }

    showApiFeedback("Obtenint la teva ubicació...", "info");

    // 2. Opcions de configuració (PositionOptions)
    const options = {
        enableHighAccuracy: true, // Demana la màxima precisió
possible
        timeout: 8000,            // Temps màxim d'espera: 8
segons
        maximumAge: 0             // No utilitza posicions
antigues en memòria cau
    };

    // 3. Crida a l'API amb callbacks d'èxit i d'error
    navigator.geolocation.getCurrentPosition(
        // Callback d'èxit
        (position) => {
            const { latitude, longitude } = position.coords;
            showApiFeedback(
                `Ubicació trobada: Lat ${latitude.toFixed(4)},
Lng ${longitude.toFixed(4)}`,
                "success"
            );
        },
        // Callback d'error (Capta denegació, falta de senyal o
timeout)
        (error) => {
```

```
            let msg = "Error desconegut en obtenir la
ubicació.";
            switch (error.code) {
                case error.PERMISSION_DENIED:
                    msg = "Has denegat el permís d'accés a la
ubicació.";
                    break;
                case error.POSITION_UNAVAILABLE:
                    msg = "La informació d'ubicació no està
disponible.";
                    break;
                case error.TIMEOUT:
                    msg = "S'ha esgotat el temps d'espera per
obtenir la ubicació.";
                    break;
            }
            showApiFeedback(msg, "error");
        },
        options
    );
}

// Vinculació de l'esdeveniment al carregar el DOM
document.addEventListener("DOMContentLoaded", () => {
    const btnGPS = document.querySelector("#btnNearbyHotels");
    if (btnGPS) {
        btnGPS.addEventListener("click", findNearbyHotels);
    }
});
```

Comprovar el correcte funcionament i simular localitzacions amb DevTools:

- **Gestió de Denegació:** Amb el permís bloquejat, prem el botó amb el codi nou: el quadre `#apiFeedback` ha de mostrar immediatament el missatge d'error en vermell: *"Has denegat el permís d'accés a la ubicació."*
- **Activar eines de simulació (Sensors):** Canvia el permís d'**Ubicació** a **Permetre**. Dins de les DevTools (`F12`), prem `Ctrl + Shift + P` (o `Cmd + Shift + P` en Mac), escriu **Sensors** i prem `Enter` per obrir el panell de simulació.
- **Simulació d'Ubicacions Reals:** A la secció *Location* del panell *Sensors*, selecciona ciutats com **Tokyo** o **London** i prem el botó **Cerca Hotels Propers (GPS)**. El quadre de la interfície mostrarà el missatge verd amb les coordenades exactes de la ciutat triada formatades a 4 decimals (`Lat ... Lng ...`).
- **Simulació de Fallada de Senyal:** Al mateix panell *Sensors*, selecciona **Location unavailable** i torna a prémer el botó. La funció capturarà el cas `error.POSITION_UNAVAILABLE` i mostrarà el missatge d'error: *"La informació d'ubicació no està disponible."*

Exercici 3.3 (ampliació): Eficiència de Recursos i Segon Pla (`Page Visibility API`)

- **Diagnòstic de l'error:** A la funció `startLiveUpdates()`, s'executa un interval de sincronització amb el servidor (`setInterval`) cada 3 segons. Aquest temporitzador continua fent consultes de xarxa de manera indefinitiva, **fins i tot quan l'usuari ha minimitzat la finestra o ha canviat de pestanya**. Això malbarata bateria, dades mòbils i recursos del servidor de l'hotel.

- **Com ho pots comprovar? (Prova pràctica):**
  1. Obre la consola de desenvolupador (F12).
  2. Canvia a una altra pestanya del navegador durant uns 15 segons.
  3. Torna a la pestanya de l'aplicació. Observa com la consola continua plenta de missatges `Comprovant estat de la reserva...` que s'han executat inútilment mentre la pàgina no era visible.

- **Tasca pràctica (Pas a pas):**
  1. **Control de cicle de vida amb `document.visibilityState`:** Escolta l'esdeveniment `visibilitychange` a l'objecte `document`.
  2. **Gestió de temporitzadors:** Atura el temporitzador (`clearInterval`) quan la pàgina passi a estat `hidden` i torna'l a iniciar quan la pàgina torni a ser `visible`:

     JavaScript

```
let syncTimer = null;

function startLiveSync() {
    if (syncTimer) return; // Evita duplicar intervals

    console.log("Sincronitzador activat.");
    syncTimer = setInterval(() => {
        console.log("Comprovant estat de la reserva amb el
servidor...");
    }, 3000);
}

function stopLiveSync() {
    if (syncTimer) {
        clearInterval(syncTimer);
        syncTimer = null;
        console.log("Sincronitzador pausat (pestanya en segon
pla).");
    }
}

// Escoltador del cicle de vida de la pàgina
document.addEventListener("visibilitychange", () => {
    if (document.visibilityState === "hidden") {
        stopLiveSync();
    } else {
        startLiveSync();
    }
});

// Inicieu la sincronització en carregar
startLiveSync();
```